

BASICS OF DATABASES

for Modern AI Applications

Class six

Practical foundations for AI engineers, data scientists & developers

Agenda & Learning Objectives

01

Introduction & Motivation

Why databases are critical in AI systems

02

Types of Databases for AI

Relational, Vector, Document, Graph & Hybrid

03

Core Database Concepts

SQL, modeling, indexing, consistency

04

Typical AI Application Patterns

RAG, Feature Stores, Logging, Memory & more

01

Introduction & Motivation

Why databases are the foundation of every reliable AI system

Why Databases Matter in AI

Data is the fuel of AI. Databases are the engine that stores, organizes, retrieves and governs that fuel.



Training & Fine-tuning

Clean, versioned datasets for model training and continuous improvement



Retrieval & Context

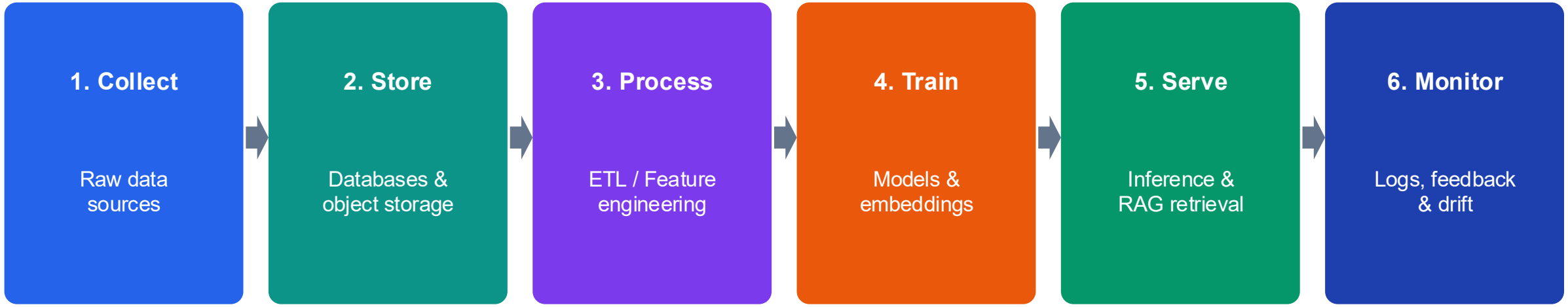
Fast semantic search over documents, embeddings and knowledge bases (RAG)



Production Systems

Feature stores, logging, monitoring, user memory and experiment tracking

AI Data Lifecycle — Where Databases Fit



Key Insight

Modern AI systems are data pipelines. Databases appear at almost every stage — from storing raw training data and features, to holding embeddings for retrieval, to logging predictions and user feedback for continuous improvement.

What Goes Wrong Without Proper Databases

Hallucinations & Poor Context

LLMs without grounded retrieval invent facts. No vector store = weak RAG.

Data Drift & Stale Models

Without versioned feature stores and monitoring tables, models degrade silently.

Unreliable Production

No transactional storage for orders, users or state leads to inconsistent AI behavior.

Impossible Debugging

Missing prediction logs and input snapshots make root-cause analysis extremely hard.

Security & Compliance Gaps

Unstructured files scattered everywhere make access control and audits painful.

Scalability Collapse

Ad-hoc CSV/JSON files break under concurrent users and growing data volumes.

02

Types of Databases Relevant to AI

Choosing the right storage for structured data, embeddings, documents and relationships

Four Core Database Families for AI



Relational

PostgreSQL, MySQL

- Structured tables & SQL
- ACID transactions
- Strong consistency
- Great for features, logs, metadata



Vector

pgvector, Pinecone, Weaviate

- Store embeddings
- Similarity search
- Core of RAG systems
- Often combined with Postgres



Document / NoSQL

MongoDB, Firestore

- Flexible JSON schemas
- Rapid prototyping
- Semi-structured data
- Good for content & configs



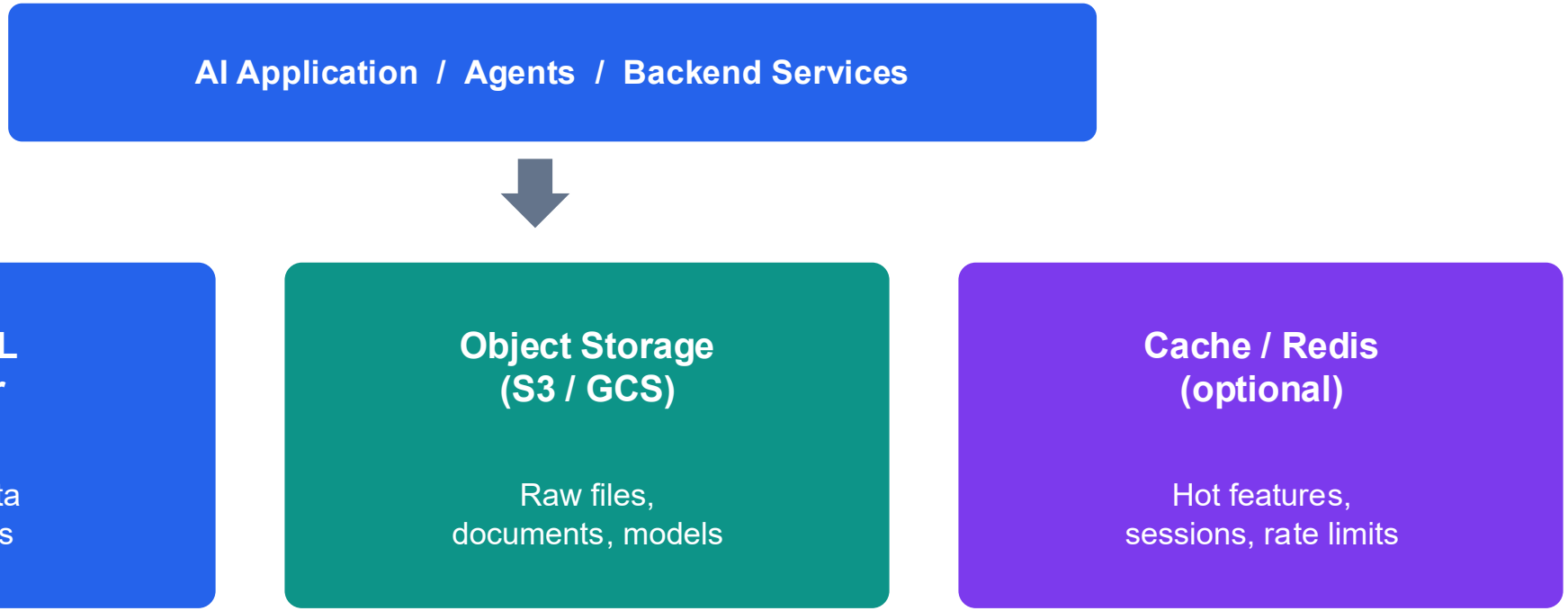
Graph

Neo4j, Amazon Neptune

- Entities + relationships
- Knowledge graphs
- Multi-hop reasoning
- Recommendation engines

The Modern Reality: Hybrid Stacks

Most production AI systems combine multiple database types. One popular and practical pattern:



Practical tip: Start with PostgreSQL + pgvector (or Supabase / Neon). It covers 80% of AI application needs — structured data, vectors for RAG, JSON support, and strong reliability — without managing multiple systems.

Practical Cloud Options for AI Applications

Supabase

Best starting point for most AI projects

What it is

Managed Postgres + Auth + Storage + Realtime + Edge Functions

AI strengths

pgvector built-in, easy Python client, free tier, Row Level Security

Best for

RAG prototypes, internal tools, startups, personal projects, MVPs

Watch out

Free projects pause after inactivity; scale limits on free tier

AWS

Enterprise scale & full AI platform

Key services

RDS / Aurora (Postgres), OpenSearch, DynamoDB, S3, Bedrock

AI strengths

Bedrock Knowledge Bases, managed vector search, high scale & compliance

Best for

Production systems, regulated industries, large data, complex pipelines

Watch out

Steeper learning curve and higher cost if not carefully designed

Quick Decision Guide

Choose the right starting point based on your stage and constraints

Learning / Prototype

- Supabase or Neon
- Local Postgres + pgvector
- Chroma / simple vector DB

MVP / Small Team

- Supabase (full backend)
- Neon + Vercel / Railway
- Pinecone for vectors only

Production / Scale

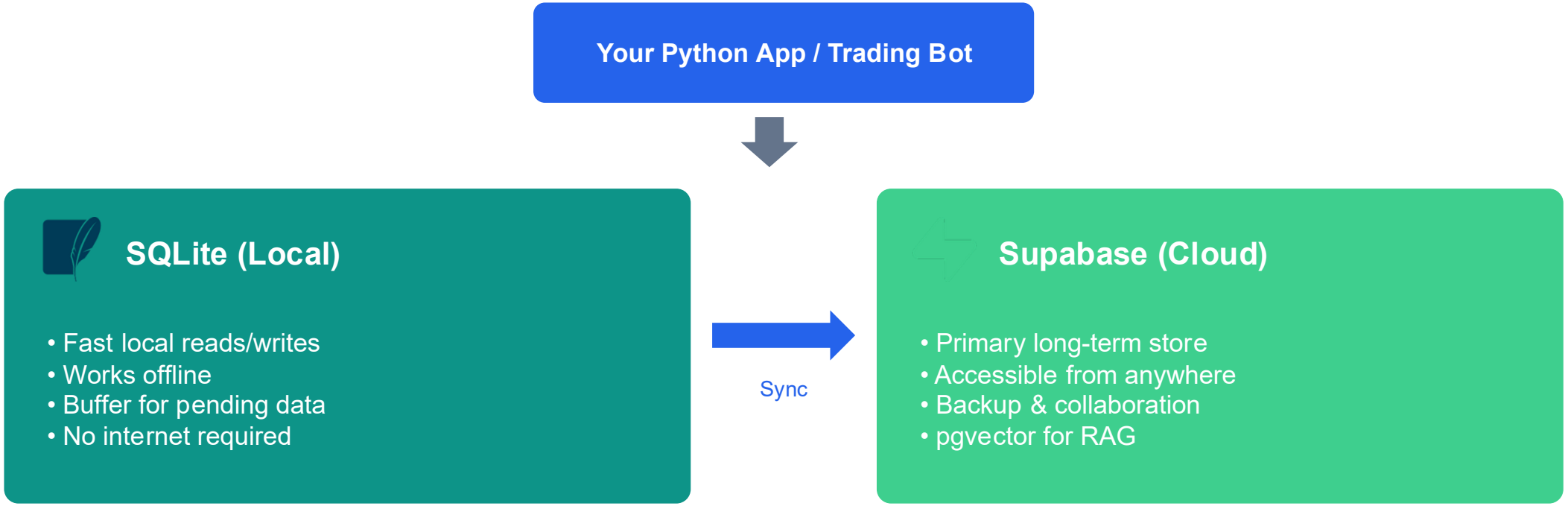
- AWS Aurora + OpenSearch
- Supabase Pro / Enterprise
- Dedicated vector DB + RDS

Enterprise / Regulated

- AWS (RDS, Bedrock, IAM)
- Self-hosted on VPC
- Strong audit & compliance focus

Local + Cloud Hybrid: SQLite + Supabase

Combine a local SQLite database with Supabase for better reliability and offline resilience.



Why this pattern works well for trading & AI apps: Local speed + offline safety + cloud durability. Write to SQLite first, then sync to Supabase when the network is available.

When & How to Use SQLite + Supabase

When to use this pattern

Unstable or limited internet

Home networks, mobile, remote locations

Trading / monitoring bots

Must keep running even if cloud is unreachable

Need very fast local queries

Recent signals, open positions, caches

Want offline-first design

Work continues; sync later when online

Simple backup strategy

Local file is easy to copy; cloud is the durable store

Typical implementation steps

1. Create the same core tables in both SQLite and Supabase
2. Write new data first to SQLite (always succeeds locally)
3. Background job or retry loop pushes pending rows to Supabase
4. Mark rows as synced; handle conflicts if needed
5. Read recent/hot data from SQLite; historical from Supabase

03

Core Database Concepts for AI Practitioners

The essential knowledge every AI engineer should have

Essential Building Blocks

Tables & Schema

Rows, columns, data types. Design tables for entities: users, documents, embeddings, predictions.

Keys & Relationships

Primary keys uniquely identify rows. Foreign keys link tables (e.g. document → chunks).

SQL Basics

SELECT, INSERT, UPDATE, DELETE, JOIN, WHERE, ORDER BY, LIMIT — enough for most AI work.

Indexing

Speeds up lookups. Critical for vector indexes (HNSW/IVFFlat) and frequent filter columns.

Consistency & ACID

Transactions protect integrity. Important when writing features + logs together.

Data Modeling for AI

Separate structured metadata from embeddings. Version datasets and features explicitly.

Example: Tables & Schema Design — Customer CRM

customers	orders	support_tickets	interactions
customer_id (PK)	**order_id (PK)**	**ticket_id (PK)**	**interaction_id (PK)**
full_name	**customer_id (FK)**	**customer_id (FK)**	**customer_id (FK)**
email (UNIQUE)	order_date	subject	channel
phone	total_amount	priority	summary
created_at	status	status	agent_id
status	channel	opened_at	created_at

Relationships (Foreign Keys)

customers 1 —— N orders | customers 1 —— N support_tickets | customers 1 —— N interactions
One customer can have many orders, tickets, and interactions. Foreign keys enforce referential integrity.

Example Data Model for a RAG Application

documents	chunks	queries_log
id (PK)	id (PK)	id (PK)
title	document_id (FK)	user_id
source	content	question
created_at	embedding (vector)	retrieved_ids
metadata (JSONB)	token_count	answer
		latency_ms

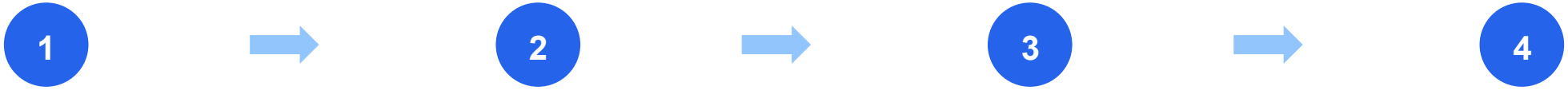
documents 1 —— N chunks | queries_log stores every RAG interaction for analysis

04

Typical AI Application Patterns

How databases are actually used in real AI systems today

Pattern 1: Retrieval-Augmented Generation (RAG)



Ingest

Split docs → embed → store vectors + metadata

Query

User question → embed → similarity search

Retrieve

Top-k chunks returned from vector DB

Generate

LLM answers with retrieved context

Database Role in RAG

- Store document chunks + metadata
- Store embedding vectors (pgvector / dedicated)
- Support fast similarity search (cosine / L2)
- Filter by metadata (source, date, access)
- Log every query + retrieved context

Common Tools

- Postgres + pgvector (simple & powerful)
- Supabase / Neon (managed Postgres)
- Pinecone, Weaviate, Chroma, Milvus
- LangChain / LlamaIndex integrations
- Hybrid search (keyword + vector)

Other High-Value AI Database Patterns

Feature Store

Store online & offline features for models

Feast, Tecton, custom Postgres

Model Registry

Track models, versions, metrics, artifacts

MLflow + DB, custom tables

Prediction Logging

Inputs, outputs, latency, feedback for audit

Postgres, ClickHouse

Chat / Agent Memory

Conversation history & long-term memory

Postgres, Redis + Postgres

Recommendation

User-item interactions + embeddings

Hybrid relational + vector

Training Data Mgmt

Labeled datasets, versions, splits

Postgres + object storage

Key Takeaways

1. Databases are not optional in serious AI systems — they power training, retrieval, serving and monitoring.

2. Know the four families: Relational, Vector, Document, Graph. Most teams start with Postgres + vectors.

3. Master basic SQL, schema design, keys and indexing. These skills transfer across tools.

4. RAG, Feature Stores, Logging and Memory are the highest-ROI patterns to learn first.

5. Use SQLite locally + Supabase in the cloud for better reliability when internet is not perfect.